

EC441 MT2 Spring 2025 (Ver 179) — Solutions

Q1 Solutions: Subnets

Part (a)

IP address: **116.50.137.187**, netmask: **255.192.0.0**

The netmask $255.192.0.0 = 11111111.11000000.00000000.00000000 = /10$.

Apply the mask:

```
116.50.137.187 = 01110100.00110010.10001001.10111011
255.192.0.0    = 11111111.11000000.00000000.00000000
AND            = 01110100.00000000.00000000.00000000
                = 116.0.0.0
```

Network address: 116.0.0.0/10

Part (b)

A **/20** network has $32 - 20 = 12$ host bits, giving $2^{12} = 4096$ total addresses. Subtract 2 (network address and broadcast): $4096 - 2 = \mathbf{4094}$ **usable hosts**.

Part (c)

Partition **127.0.0.0/9** (covering 127.0.0.0 – 127.127.255.255) into exactly 5 subnets.

Split the **/9** into four **/11** subnets (adding 2 bits gives $2^2 = 4$ equal pieces), then subdivide the last **/11** into two **/12** subnets to reach 5 total:

Subnet	Range	Prefix
127.0.0.0/11	127.0.0.0 – 127.31.255.255	/11
127.32.0.0/11	127.32.0.0 – 127.63.255.255	/11
127.64.0.0/11	127.64.0.0 – 127.95.255.255	/11
127.96.0.0/12	127.96.0.0 – 127.111.255.255	/12
127.112.0.0/12	127.112.0.0 – 127.127.255.255	/12

These five subnets are non-overlapping and together cover all of 127.0.0.0/9. (*Many other valid partitions exist.*)

Q2 Solutions: Routing

Network graph

Given: A–B (cost 6), A–D (cost 1), and A’s distance vector:

y	$D_A(y)$
A	0
B	4
C	8
D	1
E	2
F	8

Running Dijkstra from A with $D_A(E) = 2 = D_A(D) + 1$ implies D–E (cost 1). $D_A(B) = 4 = D_A(E) + 2$ implies B–E (cost 2), which is shorter than the direct A–B link. $D_A(C) = 8$ and $D_A(F) = 8$ are reached via B–C (cost 4) and E–F (cost 6) / C–F (cost 2).

The consistent graph uses edge weights:

Edge	Cost
A–B	6
A–D	1
B–C	4
B–E	2
C–F	2
D–E	1
E–F	6

(*All costs are symmetric.*)

Verification — shortest paths from A: $A \rightarrow D=1$, $A \rightarrow D \rightarrow E=2$, $A \rightarrow D \rightarrow E \rightarrow B=4$, $A \rightarrow D \rightarrow E \rightarrow B \rightarrow C=8$, $A \rightarrow D \rightarrow E \rightarrow B \rightarrow C \rightarrow F=10$ vs $A \rightarrow D \rightarrow E \rightarrow F=8$ (consistent)

Part (b): Shortest path tree from F

Dijkstra from F:

Step	Visited	Distances (A, B, C, D, E, F)
Init		$\infty, \infty, \infty, \infty, \infty, \mathbf{0}$
1	F	$\infty, \infty, \mathbf{2}, \infty, 6, 0$
2	C	$\infty, \mathbf{6}, 2, \infty, 6, 0$
3	B (tie)	$12, 6, 2, \infty, 6, 0$
4	E (tie)	$12, 6, 2, \mathbf{7}, 6, 0$
5	D	$\mathbf{8}, 6, 2, 7, 6, 0$
6	A	$8, 6, 2, 7, 6, 0$

Shortest path tree rooted at F:

F --2--> C --4--> B
F --6--> E --1--> D --1--> A

Destination	Cost	Path
C	2	F $\rightarrow$ C
B	6	F $\rightarrow$ C $\rightarrow$ B
E	6	F $\rightarrow$ E
D	7	F $\rightarrow$ E $\rightarrow$ D
A	8	F $\rightarrow$ E $\rightarrow$ D $\rightarrow$ A

Part (c): Cost of A–B link increases — Distance Vector

A notices its link cost to B has increased. A **recomputes its distance vector** using Bellman-Ford, combining its updated local link costs with the distance vectors most recently received from its neighbors (B and D). If A's distance vector changes, A **sends its updated distance vector to all its neighbors** (B and D). Neighbors will in turn update and propagate if their vectors change.

Part (d): Cost of A–B link increases — Link State

A notices its link cost to B has increased. A **floods an updated Link State Advertisement (LSA)** — announcing the new A–B cost — to all other nodes in the network. Every node that receives the updated LSA re-runs Dijkstra's algorithm on its complete (now-updated) topology map to recompute shortest paths.

Q3 Solutions: TCP

Part (a): Application layer protocol?

No, it is not possible to determine the protocol definitively. Neither port 64029 nor port 20441 is a well-known IANA-registered port. Without inspecting the payload bytes (which tshark does not show here in text mode), the application layer protocol cannot be identified.

Part (b): Client and server

The **client** initiates the connection (sends the SYN):

- **Client:** IP 10.239.37.231, port 64029
- **Server:** IP 128.197.173.25, port 20441

Part (c): Connection establishment

Packets **1, 2, 3** — the TCP three-way handshake:

1. SYN (client → server)
2. SYN-ACK (server → client)
3. ACK (client → server)

(Packet 4 is a TCP Window Update sent by the server immediately after the handshake; it is not part of connection establishment itself.)

Part (d): Connection termination

Packets **8, 9, 10, 11, 12**:

- 8: FIN-ACK (server initiates close)
- 9: ACK (client acknowledges server data through byte 494)
- 10: ACK (client acknowledges server FIN, Ack=496)
- 11: FIN-ACK (client initiates its own close)
- 12: ACK (server acknowledges client FIN)

Part (e): Application data

- **Packet 5:** client → server, Len=4 (4 bytes of application data)
- **Packet 7:** server → client, Len=494 (494 bytes of application data)

All other packets carry no application payload (Len=0).

Part (f): Minimum RTT estimate

The SYN was sent at $t = 0.000000$ s; the SYN-ACK arrived at $t = 0.006724$ s. Since the server responds to a SYN essentially immediately (no application processing), this elapsed time is a direct estimate of one round trip:

$$\text{RTT}_{\min} \approx 6.7 \text{ ms}$$

Part (g): Upper bound on transmission rate

The receive window advertised during the handshake is **131,200 bytes** (Win=131200, WS=64, so effective window = $131200 \times 64 = \sim 8$ MB; but the unscaled value shown is 131,200 bytes). Using the flow-control throughput bound:

$$\text{Rate} \leq \frac{\text{RcvWindow}}{\text{RTT}} = \frac{131,200 \times 8 \text{ bits}}{0.006724 \text{ s}} \approx 156 \text{ Mb/s}$$

This is an upper bound: the actual connection transferred very little data, so the true throughput is far lower; 156 Mb/s reflects what the window and RTT would allow, not the link line rate.

Q4 Solutions: Error Control

Codewords: 00000, 11111, 11001, 01010

Part (a): Code rate

There are $M = 4 = 2^2$ codewords, so $k = 2$ information bits. Each codeword has $n = 5$ bits.

$$R = \frac{k}{n} = \frac{2}{5}$$

Part (b): Minimum distance

All pairwise Hamming distances:

Pair	XOR	d_H
00000 vs 11111	11111	5
00000 vs 11001	11001	3
00000 vs 01010	01010	2
11111 vs 11001	00110	2
11111 vs 01010	10101	3
11001 vs 01010	10011	3

$$d_{\min} = 2$$

Part (c): Error correction/detection capability

With $d_{\min} = 2$:

- **Error detection only:** can detect up to $d_{\min} - 1 = 1$ bit error.
- **Error correction:** $\lfloor (d_{\min} - 1)/2 \rfloor = 0$ — **no correction capability.**
- No combined correction+detection improvement is possible beyond the above.

The only capability of this code is to **detect 1-bit errors**.

Part (d): Optimized code with same $(n,k) = (5,2)$

We want to choose 4 codewords of length 5 that maximize $d_{\min}$.

By the Plotkin bound, a binary $(5, d_{\min})$ code with $2d_{\min} > n$ satisfies $M \leq 2\lfloor d_{\min}/(2d_{\min} - n) \rfloor$. For $d_{\min} = 4$, $n = 5$: $M \leq 2\lfloor 4/3 \rfloor = 2$. We need $M = 4$, so $d_{\min} = 4$ **is impossible**.

The best achievable is $d_{\min} = 3$. One such code:

```
00000
11100
00111
11011
```

Verification: all pairwise distances are 3 or 4; $d_{\min} = 3$.

With $d_{\min} = 3$, this code can **detect up to** $d_{\min} - 1 = 2$ **bit errors** (always).